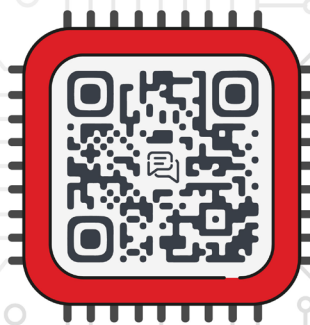




2



14



2100 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Binary Search Trees



د. مدحت الصوص

RB Informatics; 19/4/2026

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

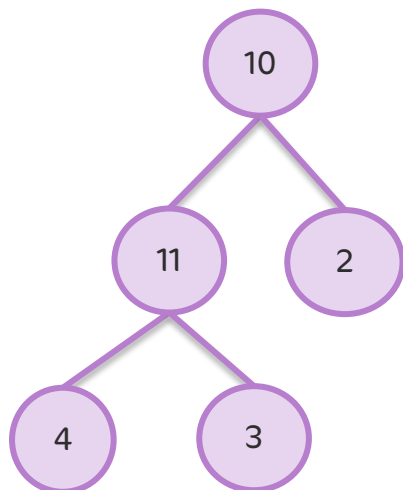
Binary Search Tree

■ الأفكار التي سوف نتعلمها في هذه المحاضرة:

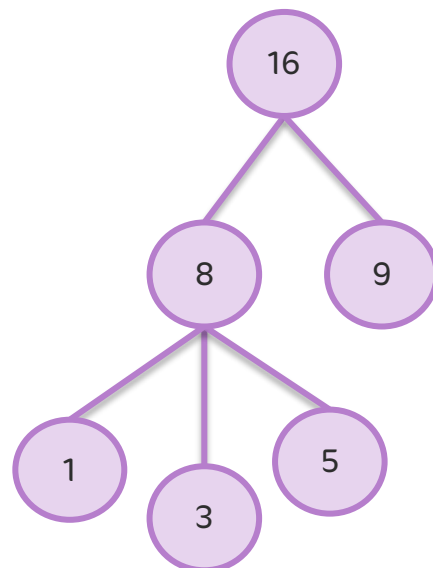
1. التعرف على Binary Search Tree (BST).
2. آلية البحث في BST.
3. الإضافة في BST.
4. الحذف في BST.
5. العنصر التالي.

🔗 تمهيد

سبق أن تعرفنا على بنية الشجرة (Tree)، وذكرنا أحد أنواعها وهي الشجرة الثنائية **Binary Tree** حيث يمكن لكل عقدة أن تمتلك على الأقل فرعين:



هذه Binary Tree



هذه ليست Binary Tree

(لوجود أكثر من فرعين للعقدة 8)

لكن عند استخدام الشجرة الثنائية للبحث عن عنصر معين (مثل العدد 5)، قد نضطر إلى المرور على عدد كبير من العقد أو حتى جميعها في بعض الحالات، مما يؤدي إلى زيادة الزمن المستغرق في البحث وارتفاع التعقيد.

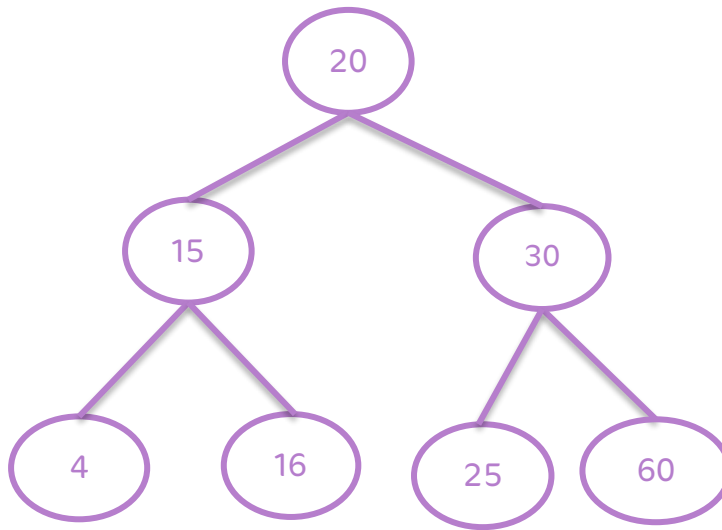
وهنا الدافع لاستخدام Binary Search Tree

- تعريفها: هي شجرة ثنائية، تخضع لقيد معين يساعد بشكل كبير في تسريع عملية البحث.
- الشرط الأساسي للقيد: لكل عقدة (Node) في الشجرة، يجب أن يتحقق ما يلي:

- جميع القيم الموجودة في الفرع الأيسر للعقدة أصغر من قيمة العقدة.
- جميع القيم الموجودة في الفرع الأيمن للعقدة أكبر من قيمة العقدة.

ملاحظة:

هذا الشرط لا يطبق فقط على الجذر (Root)، بل يجب أن يكون صحيحاً في كل عقدة داخل الشجرة.

مثال عن ال BST:

آلية البحث في BST

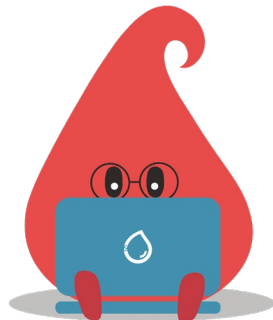
1. نبدأ من الجذر نقارن القيمة المطلوبة مع قيمة الجذر.

- إذا كانت القيمة المطلوبة أكبر من قيمة الجذر → ننتقل للبحث في الفرع الأيمن.
- إذا كانت القيمة المطلوبة أصغر من قيمة الجذر → ننتقل للبحث في الفرع الأيسر.

2. نستمر حتى نجد القيمة المطلوبة، أو نصل لعقدة فارغة (القيمة غير موجودة).

يمكن كتابة الخوارزمية السابقة بطريقتين:

- باستخدام الحل العودي.
- باستخدام الحل التكراري.



أولاً: تصميم تابع بشكل عودي:

```
1 public class Main {
2
3     private class Node {
4         int value;
5         Node right;
6         Node left;
7     }
8
9     public Node Search (Node Root, int x)
10    {
11        if (Root == null || Root.value == x)
12            return Root;           // شرط التوقف
13
14        if (x > Root.value) {
15            return Search (Root.right, x); // قيمة x أكبر من قيمة العقدة الحالية
16        }
17        else
18            return Search (Root.left, x);  // قيمة x أصغر من قيمة العقدة الحالية
19    }
```

لنكتب الكود بشكل عودي يجب أن نفكر ب:

■ العوامل:

1. الجذر (Root) ← نقطة البداية.
2. القيمة x ← التي نريد البحث عنها.

■ القيمة المعادة:

1. نعيد العقدة نفسها أو **و**
2. نعيد قيمة منطقية (True \ False).

■ شروط التوقف: نتوقف في حالتين:

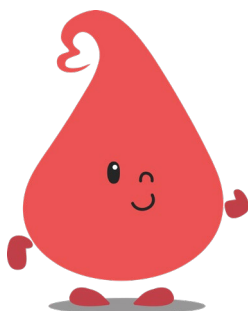
1. إذا كانت العقدة الحالية null (العنصر غير موجود).
2. إذا كانت قيمة العقدة تساوي x (تم العثور عليه).

■ منطق البحث:

1. القيمة أكبر من x ← ننتقل للبحث في الفرع الأيسر.
2. القيمة أصغر من x ← ننتقل للبحث في الفرع الأيمن.

↪ **ثانياً: البحث بشكل تكراري:**

```
1  private class Node {
2      int value;
3      Node right;
4      Node left;
5  }
6
7  public Node Search (Node node, int x) {
8
9      while (node != null && node.value != x) {
10
11         if (node.value > x)
12             node = node.left;
13         else
14             node = node.right;
15     }
16     return node;
17 }
```



**الحياة قصيرة،
لذا ابتسم مادام لديك أسنان..**

كيف نبني الحل:

1. نستخدم While، نستمر بالتكرار طالما:
 - 1- العقدة الحالية ليست null.
 - 2- ولم نجد القيمة بعد.
2. داخل الحلقة شو بصير:
 - إذا $x < \text{node.value}$... نتحرك لليساار.
 - إذا $x > \text{node.value}$... نتحرك لليمين.

التعقيد الزمني للبحث في BST:

الحالة العامة:	أسوء حالة (حالة خاصة):	
<u>بكل خطوة:</u> ▪ نحن لا نحذف عنصر واحد. ▪ نحن نحذف نصف الشجرة تقريباً.	<u>إذا كانت الشجرة خطية:</u> ▪ كل عقدة لها أبن (واحد) فقط. ← نضطر نمشي على كل عقدة.	
$O(\log_2(n))$	$O(n)$	النتيجة:

ملاحظة:

في الحالة العامة، يكون التقسيم على 2 يتكرر عدة مرات.
نحن نسأل؟ كم مرة يمكننا قسمة n على 2 حتى نصل إلى 1. (وهذا يساوي $\log_2(n)$).

الإضافة في Binary Search Tree:

حالتين في الإضافة:

1. إضافة عقدة كورقة.
2. إضافة عقدة كجذر.



لا تيأس، أستمر...

أولاً: إضافة عقدة كورقة (Leaves Add):

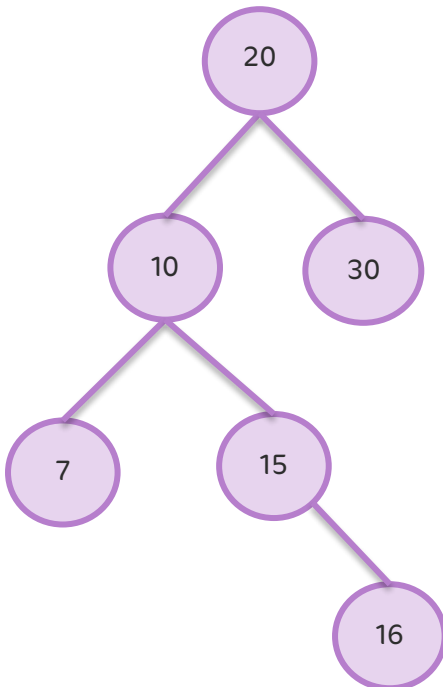
عند إضافة عقدة جديدة، نحن لا نضعها عشوائياً بل يجب أن نختار المكان الصحيح (كورقة) بحيث لا ينكسر شرط الـ BST.

آلية التفكير:

نفس منطق البحث تقريباً:

1. نبدأ من الجذر: نقارن القيمة الجديدة مع كل عقدة.
القيمة الجديدة أصغر ← نذهب لليساار.
القيمة الجديدة أكبر ← نذهب لليمين.
2. نتوقف عندما نصل إلى مكان فارغ (null)، هنا بالضبط نضيف العقدة الجديدة.

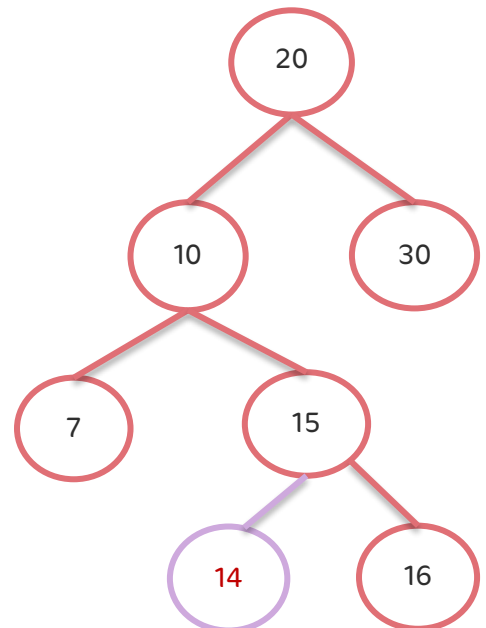
مثال للتوضيح: ليكن لدينا الشجرة التالية:



إذا اردنا إضافة العدد (14) نقارن مع كل عنصر نمر عليه، فإذا كان:

- أكبر من 14 نكمل في القسم اليساري.
- أصغر من 14 نكمل في القسم اليميني.

نستمر بالمقارنة والانتقال حتى نصل إلى عقدة (null)،
نضيف العقدة الجديدة فيه:



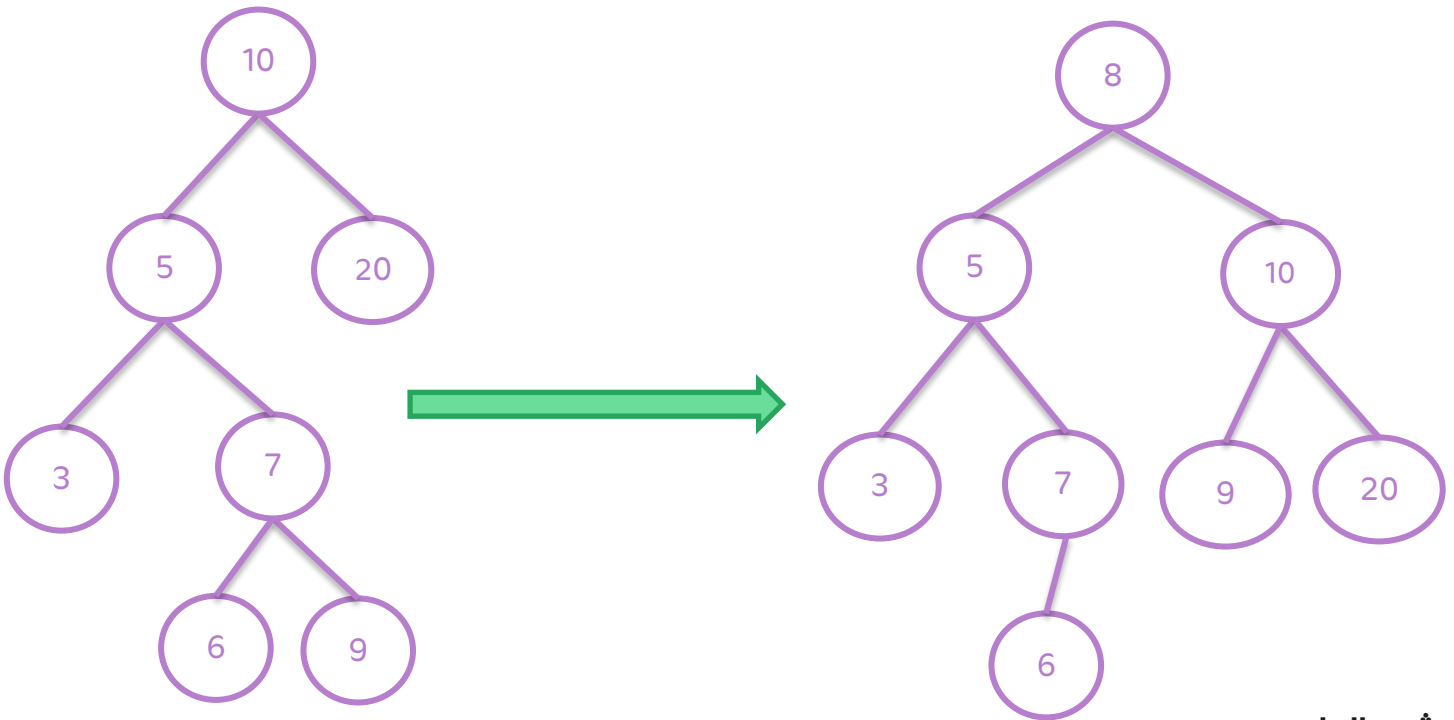
ثانياً: إضافة عقدة كجذر (Root Add):

آلية التفكير الأولى:

- نقوم بتفكيك الشجرة وإعادة تشكيلها من البداية، أي نضع الرقم الذي نريد اضافته كجذر ونضيف باقي العناصر بعده عنصراً تلو الآخر وفق مبدأ ال BST.

مثال:

لو كان لدينا الشجرة التالية وأردنا إضافة العدد (8) كجذر لهذه الشجرة:



شرح الحل:

- قمنا أولاً بوضع ال (8) كجذر، ثم بدأنا بإضافة باقي العقد واحدة تلو الأخرى وفق مبدأ ال BST.
- فمثلاً:** قمنا بإضافة ال (5) على يسار العقدة (8)، ثم أضفنا ال (7) ثم ال (3) ثم ال (10) وهكذا حتى نضيف جميع العقد.

ملاحظة: لا يوجد ترتيب محدد لإضافة العناصر إلى الشجرة، فمثلاً يمكننا إضافة ال (7) قبل ال (5).

- تعقيد هذا الحل:

نلاحظ أننا في هذه الطريقة سنضطر إلى إعادة المرور على جميع العناصر بتعقيد $O(n)$ وإعادة ترتيبها في الشجرة الجديدة بتعقيد $O(H)$ لكل عقدة، ولذلك فإن التعقيد يكون $O(n \times H)$ أي عدد العناصر $\times$ ارتفاع الشجرة، ويمكن تحسين هذا التعقيد كما سنلاحظ في طريقة الحل الثانية.

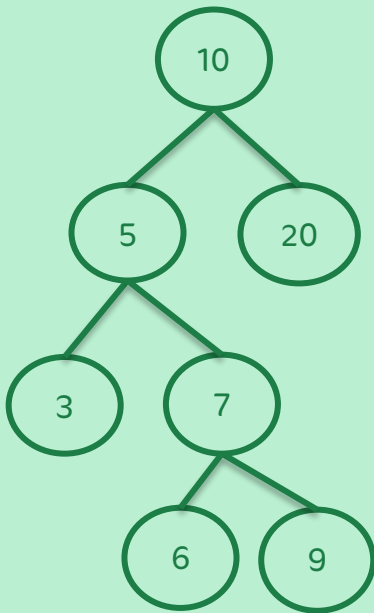


لا تستسلم.... ^_^

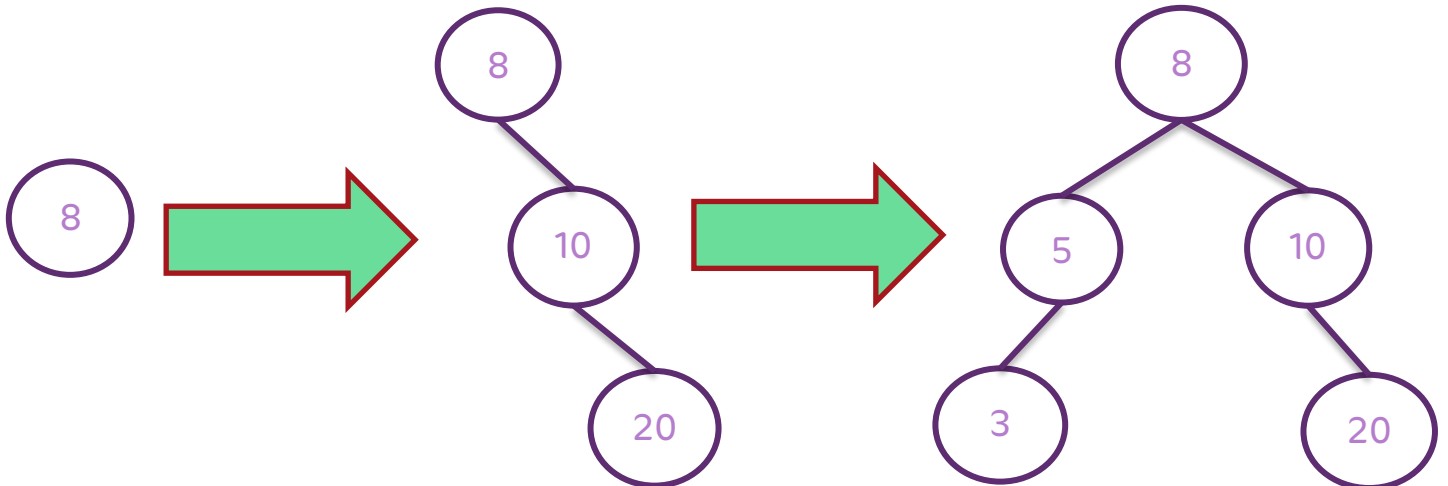
آلية التفكير الثانية:

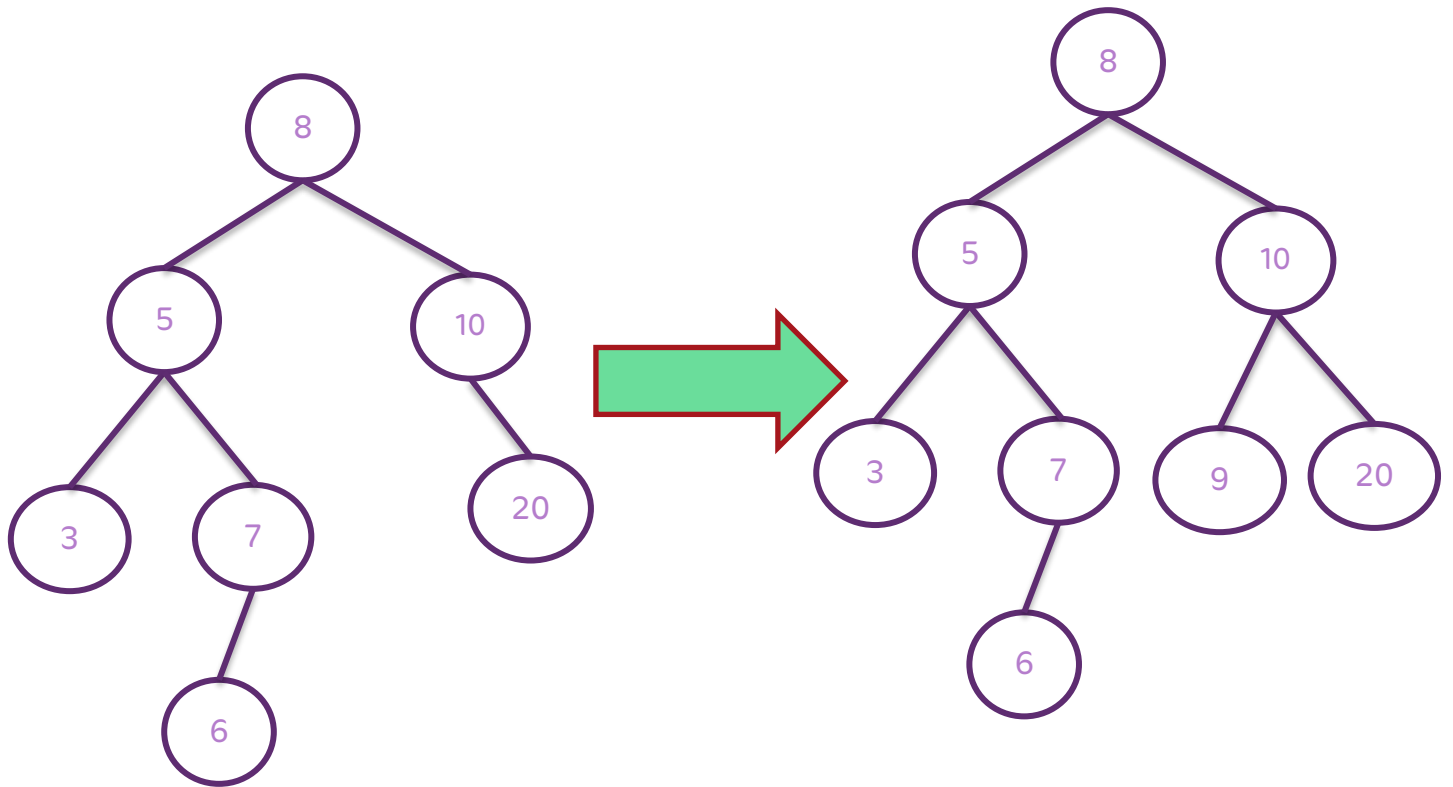
- نقارن قيمة العقدة المراد إضافتها كجذر جديد للشجرة مع قيمة الجذر السابق:
1. إذا كانت أكبر منه ننقل الجذر السابق وشجرته الفرعية من جهة اليسار (أي ال left subtree الخاصة بالجذر السابق) لتصبح ابن أيسر للجذر الجديد.
 2. إذا كانت أصغر منه ننقل الجذر السابق وشجرته الفرعية من جهة اليمين (أي ال right subtree الخاصة بالجذر السابق) لتصبح ابن أيمن للجذر الجديد.
- وهكذا يمكننا إضافة الجذر دون الحاجة للمرور على جميع العناصر ومقارنتها بالجذر الجديد.

مثال:



1. نضع ال (8) كجذر للشجرة.
2. نقارن الجذر الجديد (8) مع الجذر القديم (10) فنجد أن $10 > 8$ لذلك نأخذ ال (10) وجميع العناصر على يمينها ونضعها على يمين ال (8).
3. نقارن ال (5) مع ال (8) فنجد أن $5 < 8$ ، لذلك نأخذ ال (5) وجميع العناصر على يسارها ونضعها على يسار ال (8).
4. الآن نقارن ال (7) مع ال (8) فنجد أن $7 < 8$ ، فنذهب إلى جهة اليسار، ثم نقارن ال (7) مع ال (5) فنجد أن $7 > 5$ ، لذلك نأخذ ال (7) وجميع العناصر على يسارها ونضعها على يمين ال (5).
5. أخيرا نقارن ال (9) مع ال (8) فنجد أن $9 > 8$ ، فنذهب إلى جهة اليمين، ثم نقارن ال (9) مع ال (10) فنجد أن $9 < 10$ ، لذلك نضيفها على يسار ال (10).





ملاحظة:

- تعقيد هذا الحل هو من رتبة ارتفاع الشجرة أي $O(H)$.
- نستفيد من إضافة عقدة جديدة للجذر عندما يكون احتمال البحث عن العقدة المضافة مؤخراً أكبر، حيث تصبح عملية البحث أسرع لأنه العنصر الأقرب.

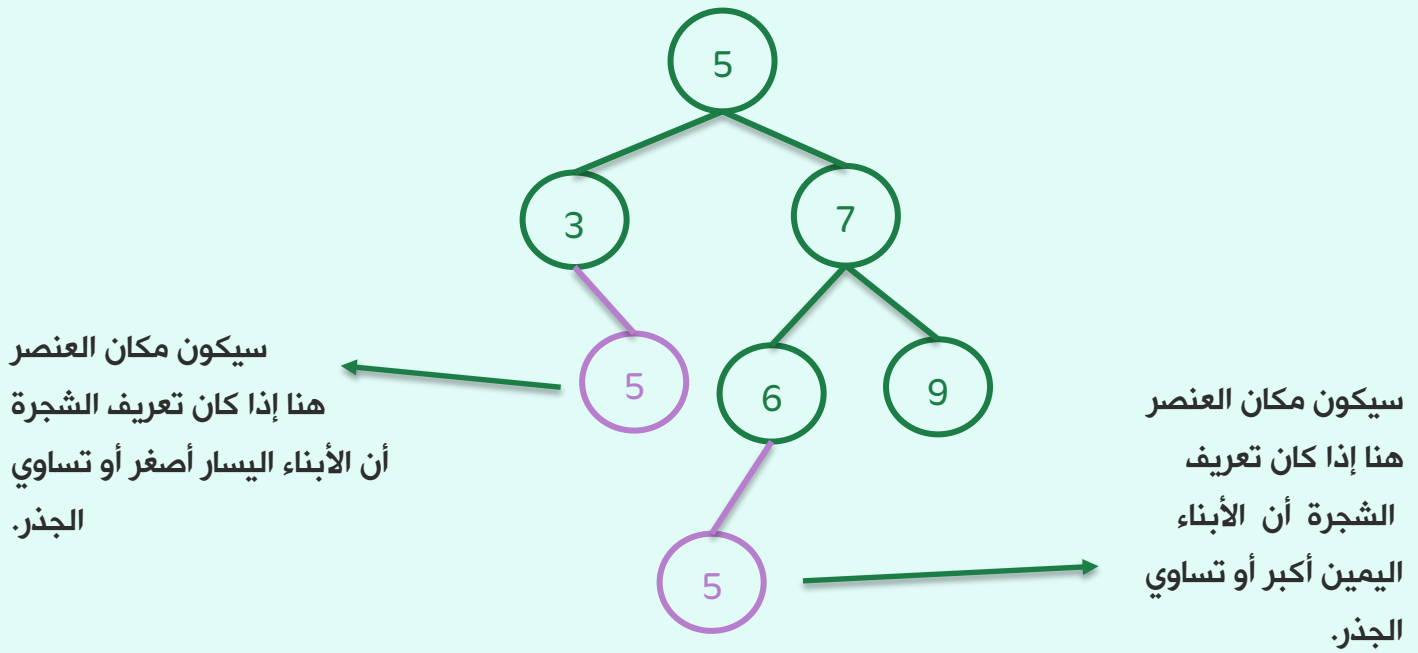
سؤال:

لو أردنا إضافة عنصر كورقة، وكان هذا العنصر موجود مسبقاً في الشجرة:

هناك حلان لهذه المشكلة:

- الحل الأول: هو أن نضع عداد في كل عقدة، فعند تكرار العنصر في الشجرة نفسها يزيد هذا العداد بمقدار التكرار لهذا العنصر.
- الحل الثاني: هو أن نعرف الشجرة بطريقة تسمح بتكرار العناصر، أي سوف نغير بنية الشجرة. بحيث يكون الأبناء اليسار **أصغر أو تساوي** الجذر والأبناء اليمين **أكبر تماماً** من الجذر أو يكون الأبناء اليمين **أكبر أو تساوي** الجذر والأبناء اليسار **أصغر تماماً** من الجذر (نحدد ذلك على رغبتنا داخل الكود).

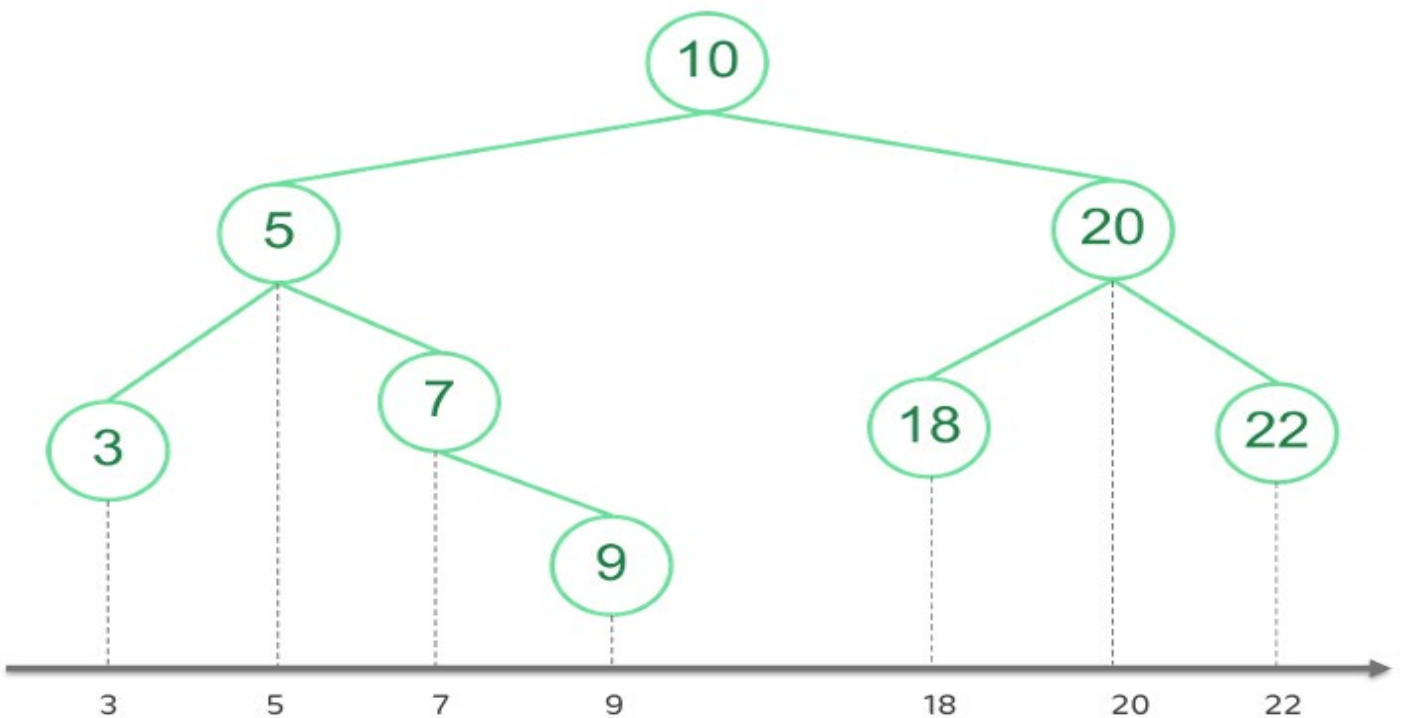
■ مثال: إذا أردنا إضافة العنصر (5) للشجرة التالية:



العنصر التالي (successor)

نقصد بالعنصر التالي لعقدة معينة بأنه العنصر الذي يليها مباشرة من حيث القيمة، أي لو قمنا بترتيب عناصر الشجرة فما هو العنصر الذي سوف يلي هذه العقدة.

للتوضيح سنقوم بإسقاط عناصر الشجرة التالية على محور الأعداد x:



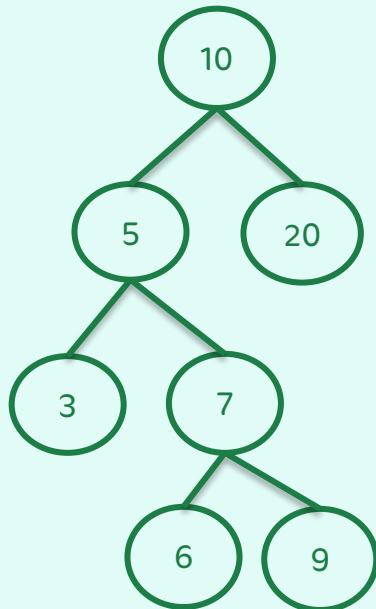
نلاحظ أن العناصر على المحور تكون مرتبة، أي العنصر الذي يلي (3) هو (5) والعنصر الذي يلي (5) هو (7)
والعنصر الذي يلي (20) هو (22).

لكن كيف يمكننا التجول في الشجرة لإيجاد العنصر التالي:

يوجد حالتين لإيجاد العنصر التالي (successor):

ليس للعقدة ابن يميني	للعقدة ابن يميني
العنصر التالي هو أول أب أكبر من هذه العقدة، أي نصعد إلى الأعلى حتى نجد أب لهذه العقدة أكبر منها، فيكون هو العنصر المطلوب.	العنصر التالي هو العقدة التي تقع أقصى اليسار في الشجرة الفرعية اليمينية لهذه العقدة. وبمعنى آخر نوجد أصغر قيمة في القسم اليميني (Right subtree) لهذه العقدة، والتي تقع على أقصى اليسار في هذه الشجرة الفرعية.

مثال: ليكن لدينا الشجرة التالية:



1. لإيجاد العنصر التالي ل (5): نلاحظ أن للعقدة ابن يميني، فنذهب أولاً إلى اليمين، ثم نستمر في الذهاب إلى اليسار حتى نصل إلى عقدة ليس لها أبناء (حتى نصل إلى الورقة)، أي سوف نصل للعقدة (6).
إذاً العنصر التالي ل (5) هو (6).
2. لإيجاد العنصر التالي ل (3): نلاحظ أنه ليس للعقدة ابن يميني، لذلك سوف نصعد للأعلى، أي سنصعد للعقدة (5) و $5 > 3$
إذاً العنصر التالي ل (3) هو (5).
3. لإيجاد العنصر التالي ل (9): نلاحظ أنه ليس للعقدة ابن يميني، لذلك سوف نصعد للأعلى، أي سنصعد للعقدة (7)، لكن $7 < 9$
وبالتالي سنصعد مجدداً للأعلى للعقدة (5)، ولكن أيضاً $5 < 9$
فنصعد مجدداً إلى ال (10) و $10 > 9$ إذاً العنصر التالي ل (9) هو (10).
4. في حالة العنصر (20) نلاحظ أنه لا يوجد أباء أكبر منه لذلك فهو أكبر عنصر ولا يوجد عنصر أكبر منه، أي لا يوجد عنصر تالي لهذه العقدة.

ملاحظة: إن تعقيد إيجاد العنصر التالي هو من رتبة ارتفاع الشجرة أي التعقيد يساوي $O(H)$.

الحذف Deletion In BST

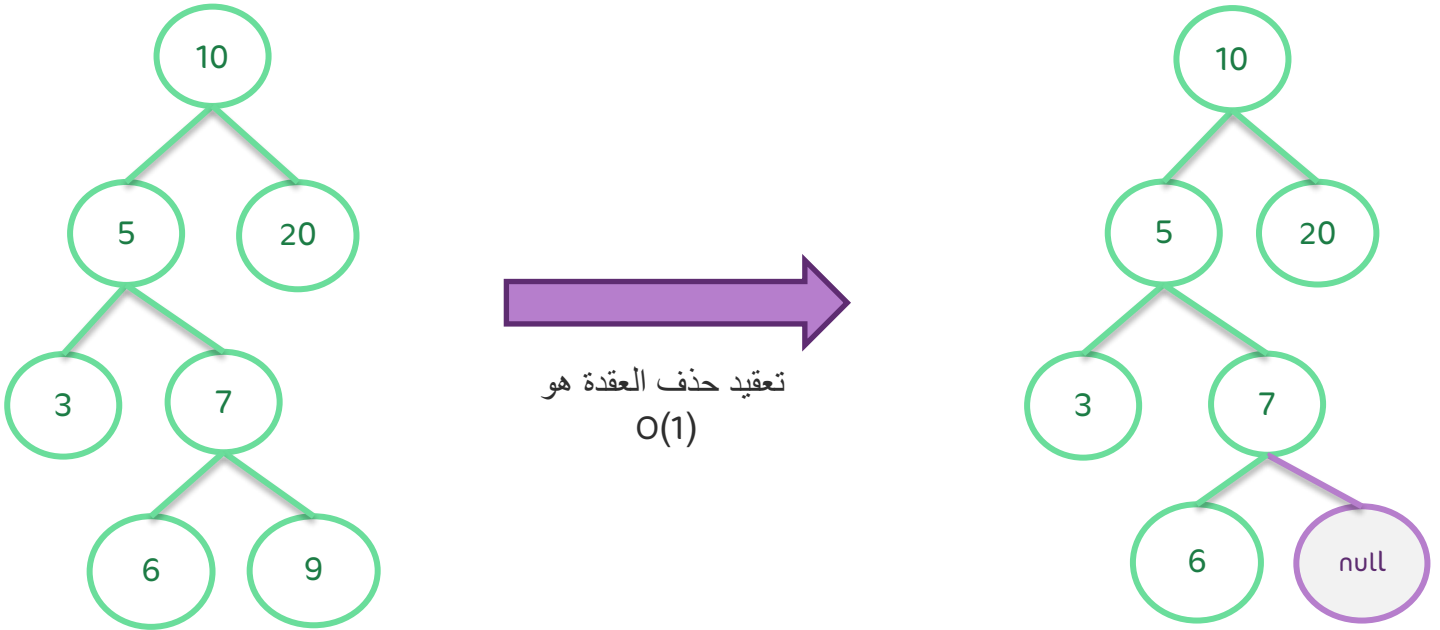
لحذف عقدة من ال BST يوجد لدينا ثلاث حالات:

1. حذف عقدة ليس لها أبناء (حذف ورقة):

إذا كان العنصر الذي نريد حذفه من الشجرة هو ورقة، فنقوم بحذفه مباشرة دون الاضطرار لتعديل الشجرة، أي نجعل العقدة الأب للورقة التي نريد حذفها تؤول إلى null.

مثال:

إذا أردنا حذف العقدة (9) من الشجرة التالية، سنجعل مؤشر الابن الأيمن للعقدة (7) يؤول إلى null.

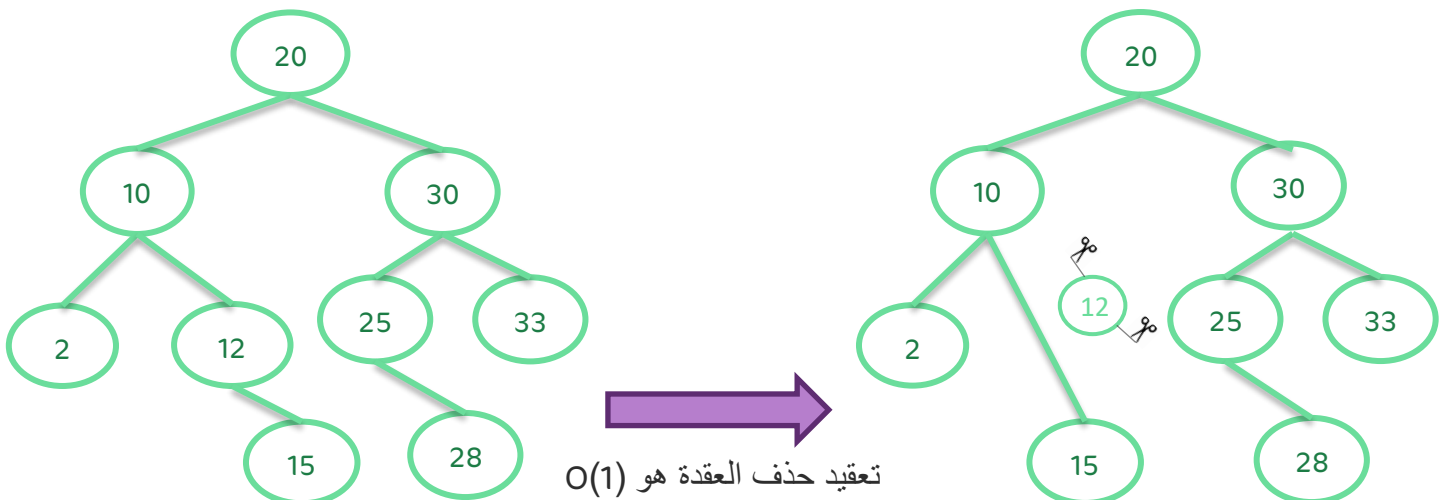


2. العقدة لها ابن وحيد (أيمن أو أيسر):

نحذف العقدة ثم نجعل مؤشر الأب يشير لابن الوحيد لهذه العقدة (قد يكون الابن شجرة فرعية أو ورقة).

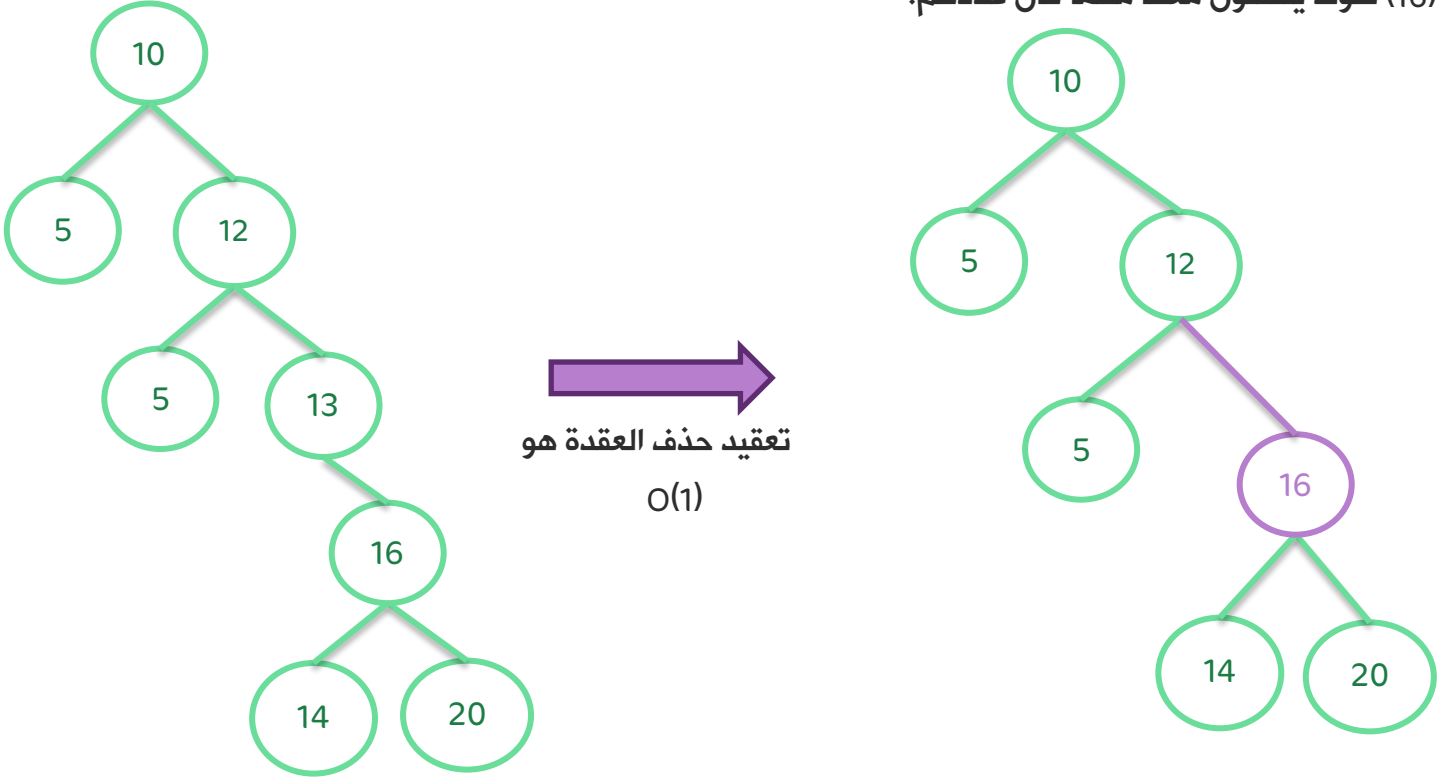
مثال:

إذا أردنا حذف العقدة (12) من الشجرة التالية نجعل المؤشر الأيمن لل 10 (parent) يؤول إلى 15.



مثال:

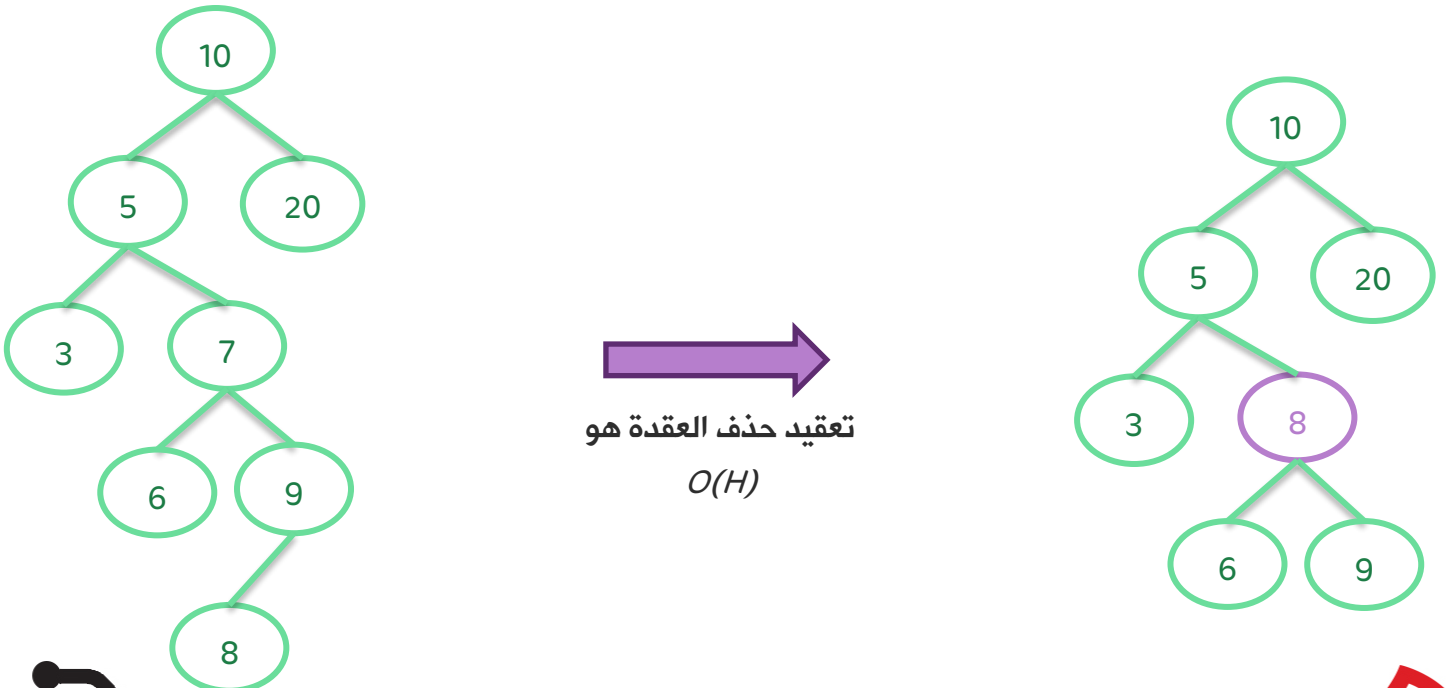
إذا أردنا حذف العقدة (13) من الشجرة التالية نجعل المؤشر الأيمن للـ 12 (parent) يؤول على (16)، وجميع أبناء العقدة (16) سوف ينتقلون معها مهما كان عددهم.

**3. حذف عقدة لها ابناء:**

نستبدل العقدة التي نريد حذفها بعنصرها التالي (successor) ثم نحذف العنصر التالي (successor) من مكانه القديم لمنع التكرار.

مثال: إذا أردنا حذف العنصر (7) من الشجرة التالية:

▪ نوجد العنصر التالي لـ (7) وهو (8) ▪ نستبدل الـ (8) مكان الـ (7) ▪ نحذف الـ (8) من مكانها السابق



مثال:

إذا أردنا حذف العنصر (5) من الشجرة التالية:

- نوجد العنصر التالي ل (5) وهو (6)
- نستبدل ال (6) مكان ال (5)
- نحذف ال (6) من مكانها السابق



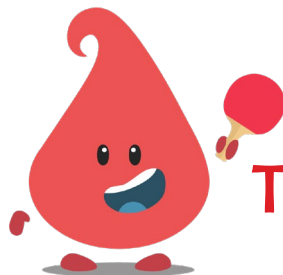
- ملاحظة: تعقيد حذف عقدة لها ابنان هو من رتبة ارتفاع الشجرة أي $O(H)$ لأننا نبحث عن العنصر التالي لعقدة ما.

تعقيد الحذف في ال BST بشكل عام:

- أولاً: نبحث عن العنصر بتعقيد $O(H)$.
 - ثانياً: نحن أمام حالتين:
 1. إما أن تكون العقدة ورقة أو لها ابن وحيد
 2. وإما أن يكون للعقدة ابنان. وفي هذه الحالة نبحث عن العنصر التالي بتعقيد $O(H)$ ، ثم نستبدل العنصر بتعقيد $O(1)$.
- باختصار:

حذف عقدة لها ابنان	حذف عقدة لها ابن وحيد	حذف ورقة
$O(H)$	$O(1)$	$O(1)$

- ملاحظة: يمكن استبدال العقدة بالعنصر السابق (predecessor) بدلاً من العنصر التالي (successor). حيث ال العنصر السابق (predecessor) هو أكبر عنصر بالفرع الأيسر للعقدة.



THE END...